

Projet Virtualisation — Examen

Conteneurisation d'un stack complet et déploiement Kubernetes en développement local

Formation professionnelle

Durée : 10 jours | Travail individuel

Enseignant : Serigne Diagne

1. Contexte et objectifs

Vous êtes chargé(e) de préparer le déploiement d'une application complète (au minimum une API + une base de données) en suivant le cycle complet vu en cours : conteneurisation avec Docker, orchestration locale avec Docker Compose, puis déploiement sur un cluster Kubernetes local (Minikube).

L'objectif est de démontrer votre maîtrise de :

- L'écriture de Dockerfile optimisés (multi-stage, utilisateur non-root, healthcheck)
- L'orchestration multi-conteneurs avec Docker Compose (réseaux, volumes, dépendances)
- Le déploiement Kubernetes : Deployment, Service, ConfigMap, Secret
- Les mécanismes clés de Kubernetes : rolling update et self-healing

2. Stack applicative

Vous êtes libre de choisir votre stack, à condition qu'elle comporte au minimum :

- Un service applicatif (API) : Node.js/Express, Java/Spring Boot, PHP/Laravel... au choix
- Une base de données : MongoDB, PostgreSQL ou MySQL au choix
- Optionnel (bonus) : un service front-end séparé (Angular ou simple HTML/JS), ou un cache (Redis)

Si vous n'avez pas de projet personnel à réutiliser, vous pouvez partir d'une petite API CRUD simple (ex : gestion de tâches, carnet d'adresses). La complexité fonctionnelle n'est pas le critère évalué, c'est la conteneurisation et l'orchestration qui comptent.

3. Travail demandé

Partie 1 : Conteneurisation avec Docker

- Écrire un Dockerfile pour le(s) service(s) applicatif(s), avec :
 - Une image de base légère (ex : alpine)
 - Un multi-stage build si le service nécessite une étape de build (npm run build, etc.)
 - Un utilisateur non-root (USER)
 - Une instruction HEALTHCHECK
 - Un **.dockerignore** pertinent
- Construire l'image et la tester seule avec docker run avant de passer à la suite
- Pousser l'image sur Docker Hub (ou un registre privé) avec un tag de version (ex : v1.0.0)

Partie 2 : Orchestration avec Docker Compose

- Écrire un **docker-compose.yml** qui démarre l'ensemble de la stack (API + BDD [+ front/cache])
- Utiliser un réseau personnalisé (pas le bridge par défaut)
- Utiliser un volume nommé pour la persistance de la base de données
- Définir **depends_on** et un **healthcheck** pour au moins un service
- Utiliser un fichier **.env** pour les variables sensibles (mots de passe, etc.), non commité dans Git

- Vérifier que **docker compose up** démarre toute la stack et que l'application est accessible et fonctionnelle

Partie 3 : Déploiement Kubernetes local avec Minikube

- Démarrer un cluster Minikube local
- Écrire les manifestes YAML suivants (dans un dossier **k8s/**) :
 - Deployment pour l'API, avec au moins **2 réplicas**, une stratégie **RollingUpdate (maxSurge/maxUnavailable)** et un **readinessProbe + livenessProbe**
 - Deployment (ou **StatefulSet simple**) pour la base de données, avec un volume persistant (PVC)
 - Service de type **ClusterIP** pour la base de données (accès interne uniquement)
 - Service de type **NodePort** pour l'API (accès depuis l'extérieur du cluster)
 - **ConfigMap** pour la configuration non sensible (ex : variables d'environnement de l'API)
 - Secret pour les données sensibles (ex : mot de passe de la base de données), injecté dans le Deployment
- Déployer l'ensemble avec **kubectl apply -f k8s/** et vérifier l'accès à l'application via minikube service
- Démontrer et documenter (captures d'écran + commandes) :
 - Le **self-healing** : supprimer un Pod et observer sa recréation automatique
 - Un **rolling update** : passer l'image de l'API à une nouvelle version (v2) et suivre **kubectl rollout status**
 - Un **rollback** : **kubectl rollout undo** et vérifier le retour à la version précédente
 - Le **scaling** : **kubectl scale deployment --replicas=4** et vérifier le nombre de Pods

Partie 4 : Rapport

Un document (Markdown ou PDF, 6 à 10 pages) contenant :

- Une présentation rapide de l'application et de son architecture (schéma simple accepté)
- Les choix techniques justifiés (images de base, stratégie de tagging, etc.)
- Les captures d'écran des tests de rolling update, self-healing, rollback et scaling
- Les difficultés rencontrées et comment elles ont été résolues
- Les commandes clés utilisées, avec une brève explication de chacune

4. Livrables

- Un dépôt Git (public ou accès partagé à l'enseignant) contenant :
 - Le code source de l'application
 - Le(s) Dockerfile
 - Le docker-compose.yml (.env.example, sans les vrais secrets)
 - Le dossier k8s/ avec tous les manifestes YAML
 - Le rapport README.md

5. Rappels utiles

Commandes clés déjà vues en cours et TP :

```
docker build -t user/app:v1 .
docker push user/app:v1
docker compose up -d

minikube start --driver=docker
```

```
kubectl apply -f k8s/  
kubectl get pods -w  
kubectl delete pod <nom>           # test self-healing  
kubectl set image deployment/api api=user/app:v2  # rolling update  
kubectl rollout status deployment/api  
kubectl rollout undo deployment/api  # rollback  
kubectl scale deployment/api --replicas=4  
minikube service api-svc --url
```

6. Consignes

- Travail strictement individuel : toute similarité anormale entre deux rendus (code, manifestes, rapport) sera considérée comme suspecte et traitée selon le règlement en vigueur.
- L'usage d'outils d'IA pour comprendre un concept est toléré, mais le code et les manifestes doivent être compris et adaptés par vous-même : des questions orales de vérification pourront être posées lors de la soutenance.
- Toute application déjà utilisée dans un TP précédent doit être clairement indiquée comme telle dans le rapport.